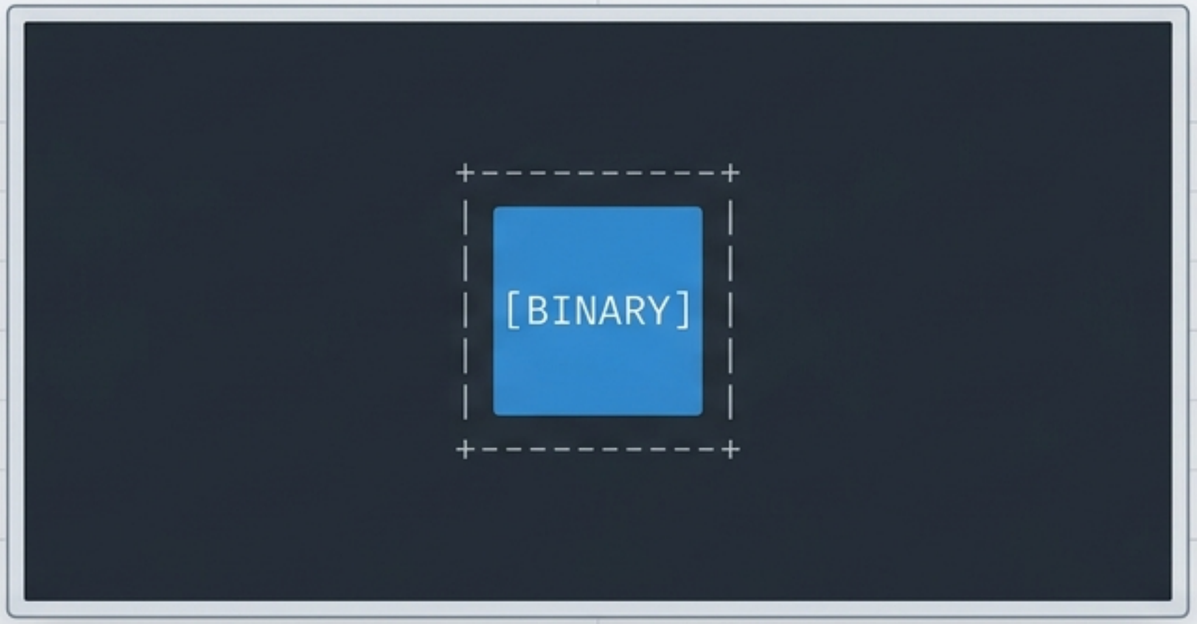


A Tiny, Single-Binary Maven Repository Server

Zero dependencies. Standard-library Go. Built for the sweet spot between a static file server and a heavyweight enterprise manager.

```
$ ./mavenrepo -addr :8080 -root /opt/repo
```

Gemini Notebook

Finding the architectural sweet spot for internal repositories

Generic Web Servers

(e.g., Nginx, Apache)

- Lacks native Maven context
- Cannot auto-generate `maven-metadata.xml`
- Requires complex manual setup for basic artifact hosting

The hub Niche



- Single 6MB compiled binary
- Natively understands Maven metadata
- Zero external dependencies
- Runs securely with simple Basic Auth

Enterprise Managers

(e.g., Sonatype Nexus, Artifactory)

- Massive JVM memory footprints
- Requires dedicated external databases
- Complex RBAC and constant maintenance overhead

Gemini Notebook

Engineered for extreme operational simplicity

Language

Pure Go, utilizing only the standard library. No third-party packages to audit or update.

Compilation

Cross-compiles natively for target servers without requiring toolchains on the host.

```
GOOS=linux GOARCH=arm64
go build
```



Containerization

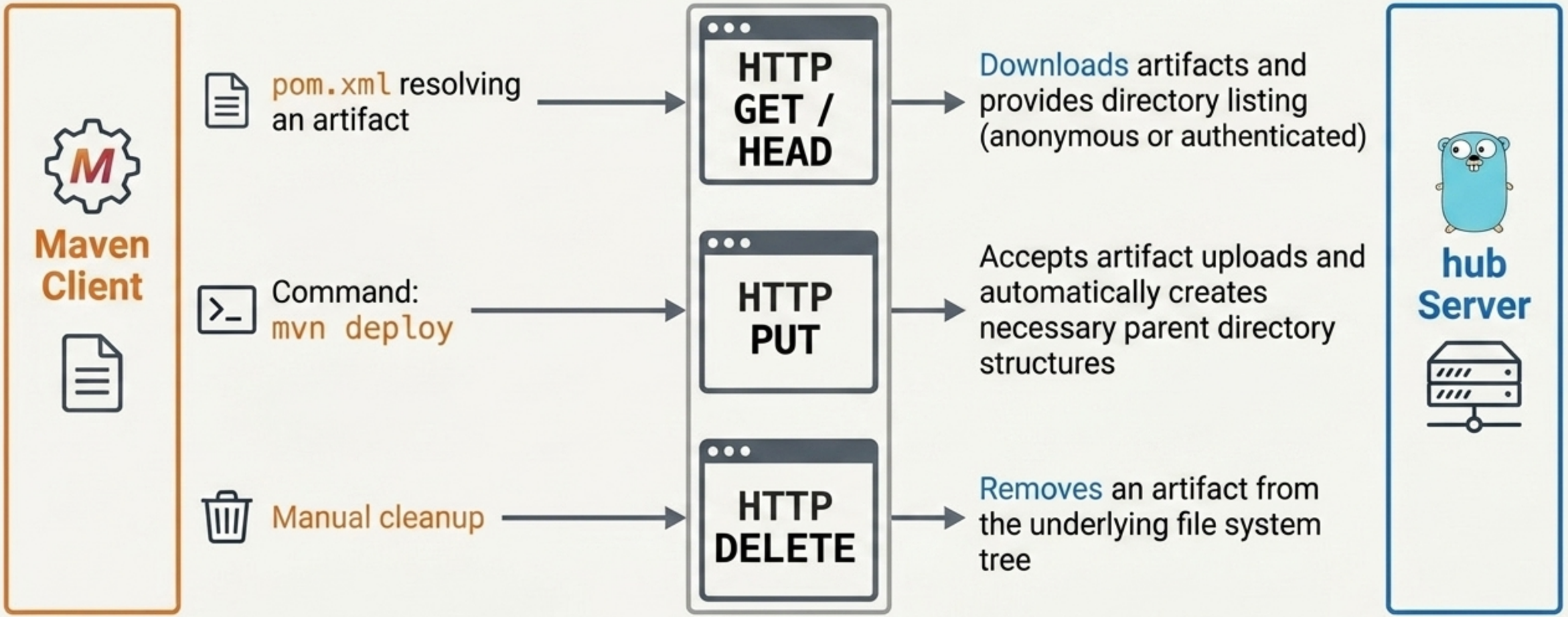
Ships as a multi-stage FROM scratch Docker image.

~6 MB

Total footprint. Runs as a non-root numeric user with no underlying OS required.

Gemini Notebook

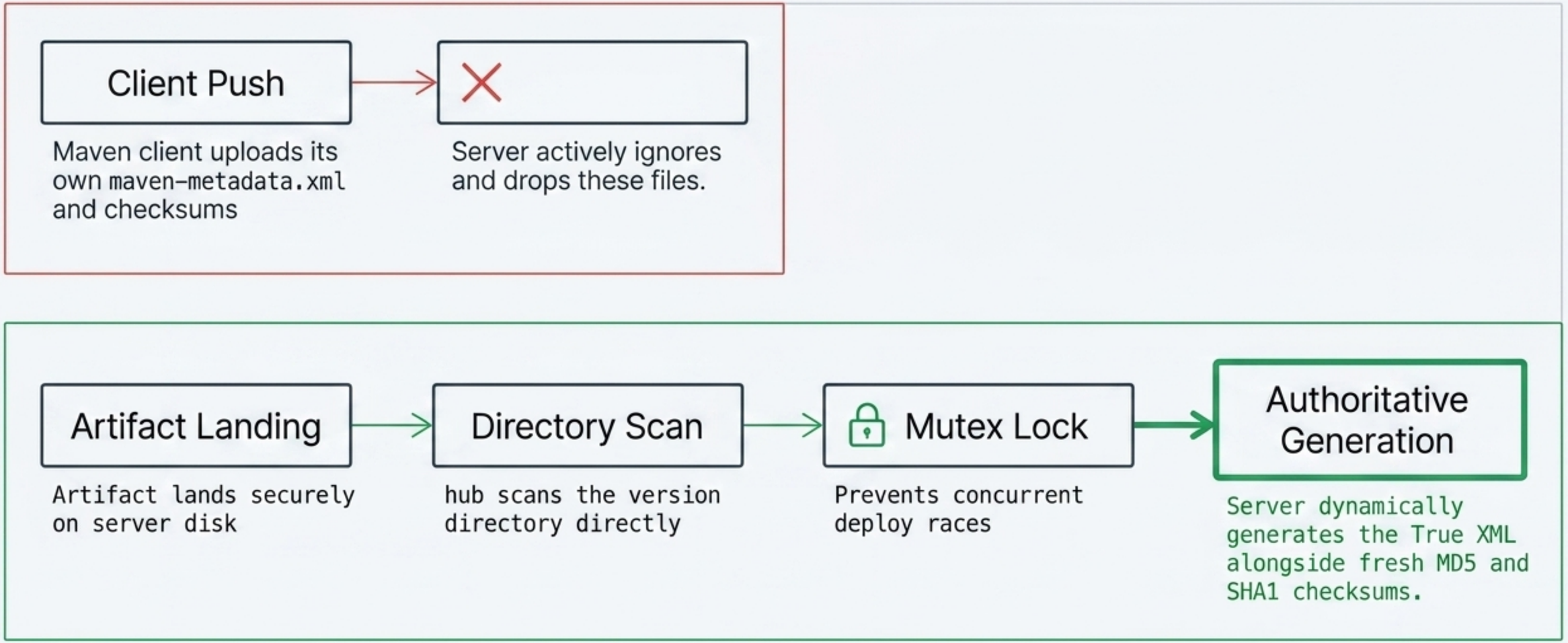
Translating HTTP primitives into Maven repository actions



Gemini Notebook

The Authoritative Metadata Engine

hub never trusts the client's version of reality. It rebuilds it from the disk.



Gemini Notebook

Native resolution for Snapshot builds

```
repo-root/
└── com/example/app/1.0-SNAPSHOT/
    ├── app-1.0-20231024.081501-1.jar
    ├── app-1.0-20231024.081501-1.pom
    ├── app-1.0-SNAPSHOT.jar (literal)
    └── maven-metadata.xml (rebuilt)
```

Disk-Based Reconstruction

The **server** rebuilds the `<snapshot>` timestamp, `<buildNumber>`, and `<snapshotVersions>` list directly from the timestamped artifacts physically present on disk.

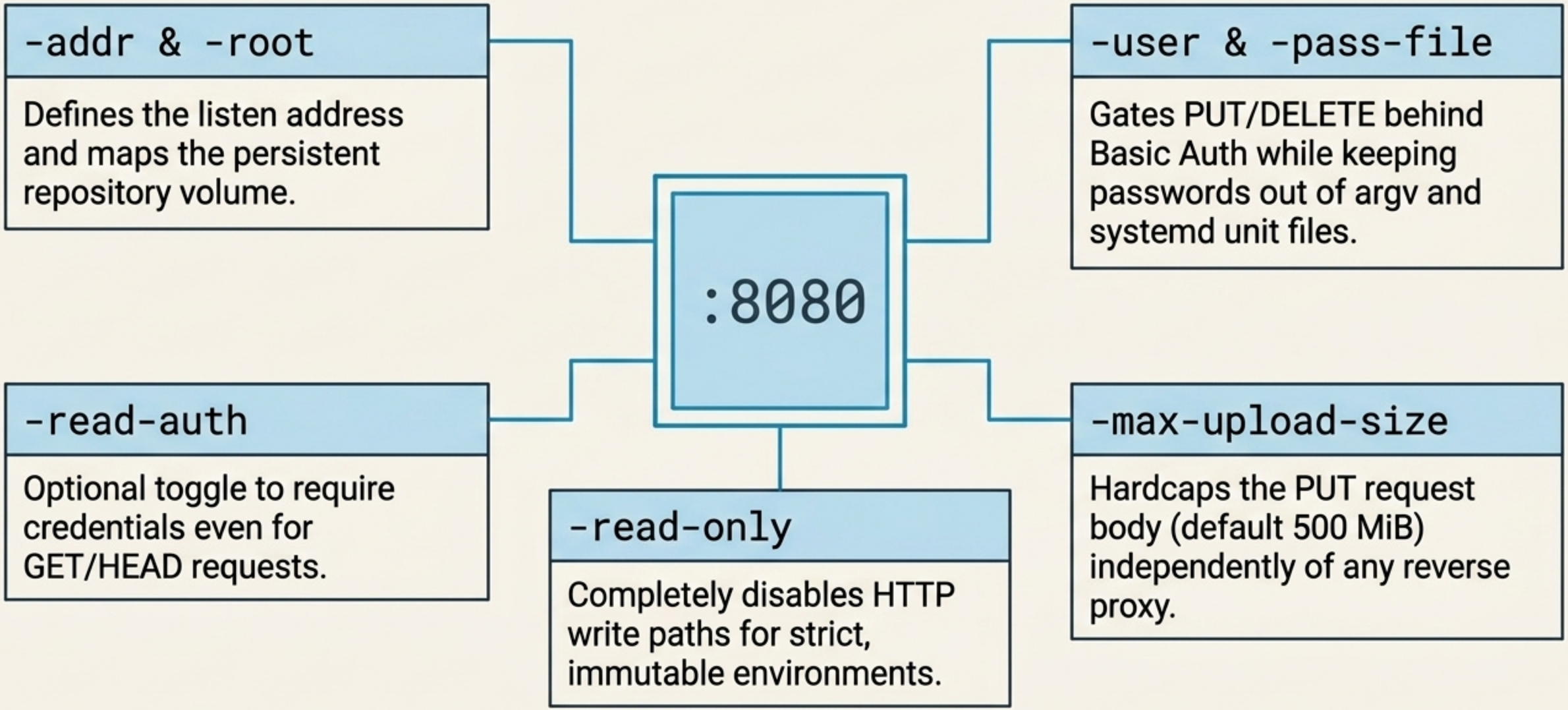
```
<version>1.0-SNAPSHOT</version>
<snapshot>
    <timestamp>20231024.081501</timestamp>
    <buildNumber>1</buildNumber>
</snapshot>
```

The Outcome ✅

`mvn deploy` of `-SNAPSHOT` builds resolves perfectly, and the build number advances automatically without requiring external state management.

Gemini Notebook

Molding the binary to your environment



Gemini Notebook

Enforcing immutable states with Read-Only Mode

HTTP: -read-only

Outright rejects all PUT/DELETE requests (HTTP 405) regardless of credentials.

Sidesteps CI/CD credential rotation overhead entirely.



Disk Storage



The Rebuild

`mavenrepo -regen <path>`

One-shot CLI command fired without starting the server.

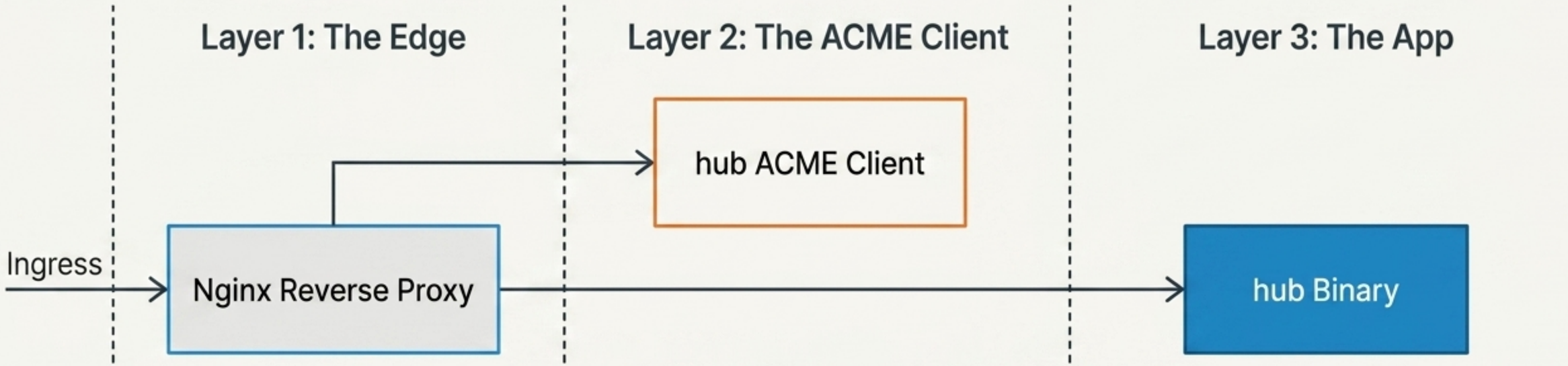
Manually rebuilds `maven-metadata.xml` based on the newly placed files, acting as the sole, non-racing writer.

Disk: Out-of-band Population

Artifacts are dropped directly into storage (e.g., vetted 3rd-party binaries placed via Ansible playbooks).

Gemini Notebook

Best practice production topology



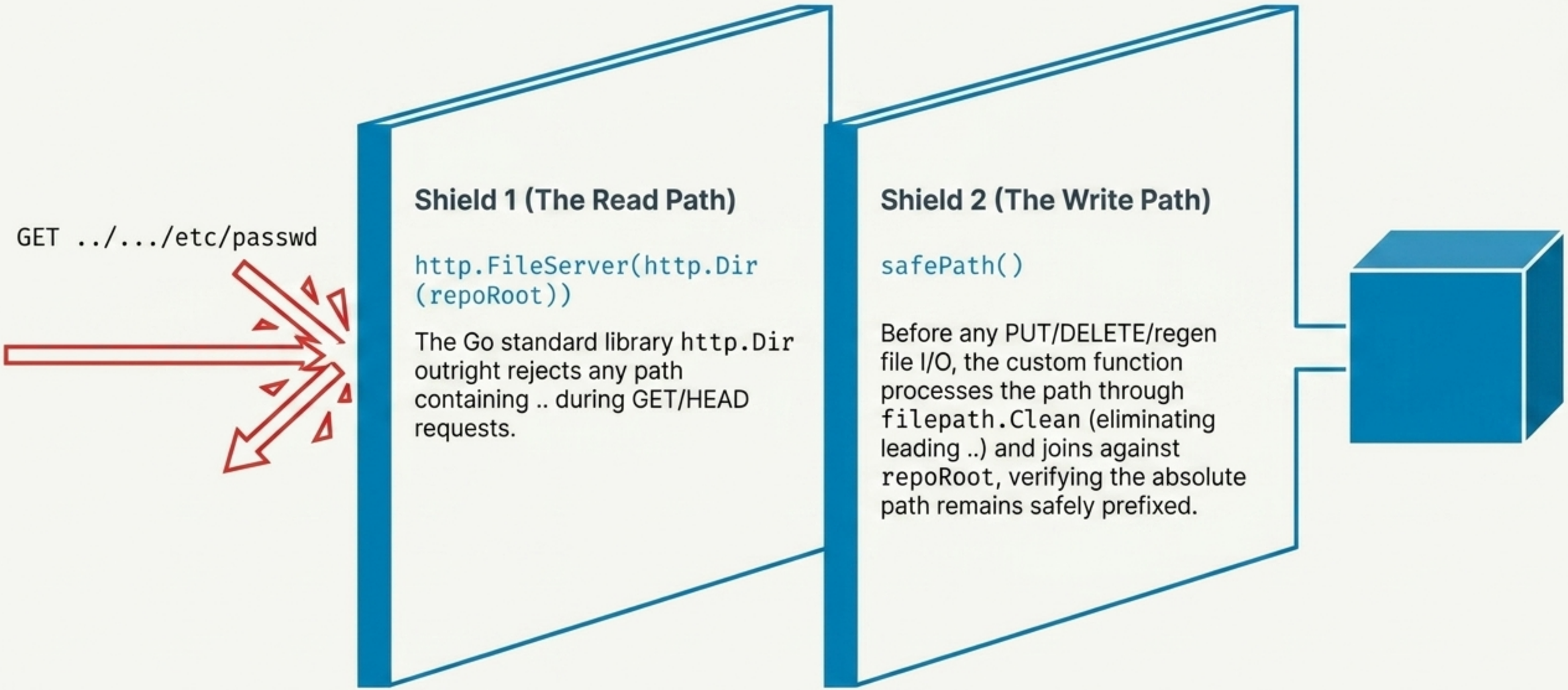
External traffic hits Nginx. Handles complete TLS termination.

Built-in, stdlib-only ACME (RFC 8555) client communicates with an internal CA to manage and auto-renew TLS certificates without Python/Certbot.

Traffic passes to hub. Nginx limits access (limit_except) to mirror application-level read-only configurations, ensuring write requests never even reach the Go backend.

Gemini Notebook

Two-layer defense against path traversal



Gemini Notebook

Hardened against application-layer abuse



Connection Holding

Mitigates Slowloris attacks. Explicit `ReadHeaderTimeout` and `IdleTimeout` guard against sockets being held open by slow-trickling headers. (**ReadTimeout** is intentionally unset to allow large artifact transfers).



Memory Exhaustion

App-layer body capping via `http.MaxBytesReader`. Aborts requests over 500 MiB with a 413 error, ensuring no partial files are left on disk.



Timing Side Channels

Uses `crypto/subtle` for constant-time credential comparison, preventing attackers from guessing basic auth passwords via response timing observation.

Gemini Notebook

The Repository Manager Diagnostic Matrix

	Generic Web Servers (Nginx)	**hub**	Enterprise Managers (Nexus)
Dependencies	None / System	Zero / Single Binary	JVM, DB, Heavy
– Metadata Handling	None / Manual	Authoritative auto-rebuild from disk	Complex dynamic caching
Access Control	Basic Auth	Basic Auth / Strict Read-Only Mode	Fine-grained RBAC / SSO
– Proxying Maven Central	✕ No	✕ No	✓ Yes
Conclusion (Best For)	Static web files	Small/medium internal artifact hosting	Global enterprise dependency proxying

Gemini Notebook